

Identifying Friend or Foe

A Face-Detection Sentry System in Python with OpenCV

Project Report

Author

Titouan SIMON

Thomas CALMEIN

Python Programming

Politechnika Krakowska

Radosław Kycia

Date: 29/05/2026

Table of Contents

Table of Contents	2
1. Abstract	3
2. Introduction	3
2.1 Aim	3
2.2 Scope	3
2.3 Methodology and Tools	4
3. Theoretical Background	4
3.1 Images as Numbers, and Why Grayscale	4
3.2 Haar-Like Features	4
3.3 The Cascade Classifier	5
3.4 Multi-Scale Detection and Its Parameters	5
3.5 Two-Stage Verification: Reducing False Positives	5
4. Implementation and Results	5
4.1 Project Structure	6
4.2 The Detection and Decision Pipeline	6
4.3 Detection in Practice	7
4.4 Decision and Feedback	7
4.5 Real-Time Webcam Mode	7
4.6 Results	7
4.7 Discussion	8
5. Summary	8
6. Bibliography	9

1. Abstract

Picture an automated sentry in a sci-fi corridor: something moves, the machine looks, and in a fraction of a second it has to decide — is this a friend who should be waved through, or a threat that should be fired upon? This project builds a small, playful version of exactly that decision, using nothing more than a webcam-era technique from classical computer vision.

Identifying Friend or Foe is a Python program that uses OpenCV to detect human faces in images (and, optionally, in a live webcam feed). To avoid being fooled by random patterns, it does not stop at finding a face: it then looks inside that face for eyes, and only a face that also contains eyes counts as a real human. Based on that result the system makes a theatrical decision — if it sees a human it stands down and plays a calm tone; if it sees no human it “fires”, stamping FIRE! on the image and playing a gunfire sound, all wrapped in a spoken Portal-style warning.

This report walks through the whole idea: why a sentry was a fun way to learn computer vision, the theory behind Haar cascade classifiers and the two-stage face-then-eyes verification, and how the program is actually structured into small, reusable pieces. It also presents the results, discusses the well-known limitations of this classical approach, and lays out how the project could grow toward modern deep-learning detection.

2. Introduction

2.1 Aim

The aim of the project was to learn the fundamentals of computer vision by building something concrete and fun: an automated “sentry gun” that can tell a person from an empty scene and react accordingly. The serious goal hiding behind the playful framing was to understand how a computer can be made to find a human face in a raw image, and how to make that detection more trustworthy than a single naive guess.

We deliberately chose the classical, pre-deep-learning approach — Haar cascade classifiers — because it is exactly the kind of technique that teaches the underlying ideas of feature extraction and cascaded classification, while being light enough to run on any laptop without a GPU [1]. The theme itself is a nod to the sentry turrets from the game Portal, which is why the program greets you with a spoken warning before it decides your fate.

2.2 Scope

The project is a self-contained command-line program. The scope we covered includes:

- Loading a sequence of grayscale images from a folder and processing them one by one.
- Detecting human faces in each image using OpenCV’s pre-trained frontal-face Haar cascade.
- Verifying each detected face by searching inside it for eyes with a second Haar cascade, which sharply reduces false positives.
- Making a binary decision from the result: a human (face with eyes) means “stand down”, while no human means “fire”.

- Giving audio and visual feedback: spoken warnings via text-to-speech, a calm tone or a gunfire sound, and the word FIRE! drawn onto the image, with detection boxes and circles around the eyes.
- An optional real-time mode that runs the same detection on a live webcam stream until the user quits.
- A small self-installing helper that checks for the required libraries and installs any that are missing, including platform-specific extras for Windows and macOS.

Out of scope, by choice, were: deep-learning detectors, tracking objects across frames, recognising who a specific person is (identity), and any actual hardware — the “gun” is entirely simulated through sound and on-screen text.

2.3 Methodology and Tools

The program is written in Python 3 and built around OpenCV (the cv2 library), the standard open-source toolkit for computer vision, which ships the pre-trained Haar cascade files we rely on [2]. Audio feedback uses two libraries: pyttsx3 for offline text-to-speech (the spoken warnings) and the Pygame mixer for playing the tone and gunfire sound files [3]. A small requirements list pins these dependencies, and a helper script installs anything missing before the program runs.

Methodologically, we worked the way the problem naturally splits: first get a single image to load and detect a face, then add the eye-verification step to cut down false alarms, then layer the decision logic and the audio-visual feedback on top, and finally add the live webcam variant. We kept the code organised by responsibility rather than dumping everything into one file: a configuration object holds all the settings and asset paths, a classifier helper wraps the OpenCV cascades, an image-tools helper handles reading images and drawing results, and a thin main script ties them together. The tools were intentionally modest — a text editor, Python, and pip — which is what made the project a good vehicle for actually understanding the computer-vision concepts rather than hiding them behind a framework.

Part I — Theoretical Part

3. Theoretical Background

To build the sentry we first had to understand a few ideas from classical computer vision. This part explains them in plain language.

3.1 Images as Numbers, and Why Grayscale

A digital image is just a grid of numbers: each pixel has a brightness (and, in colour images, three values for red, green, and blue). Face detection does not care about colour — it cares about patterns of light and dark, such as the fact that the eye region is usually darker than the cheeks below it. So the very first thing the program does with every image is convert it to grayscale, throwing away colour to keep only brightness. This makes detection both faster and more reliable, and it is why every detection in this project works on a single-channel grayscale image.

3.2 Haar-Like Features

The detector is built on Haar-like features: simple rectangular patterns that compare the total brightness of one region to a neighbouring one. For example, one feature might capture “the horizontal strip across the eyes is darker than the strip across the cheeks”. Each individual feature is almost useless on its own, but a face produces a characteristic combination of hundreds of them. A clever trick called the integral image lets the computer compute the brightness of any rectangle in constant time, which is what makes scanning a whole image at many positions and sizes fast enough to be practical [1].

3.3 The Cascade Classifier

Checking every feature at every position would still be slow, because the overwhelming majority of image regions are obviously not faces. The Viola–Jones method solves this with a cascade: a series of increasingly strict tests arranged in stages [4]. A region must pass stage one to be looked at by stage two, and so on. The vast majority of non-face regions are rejected in the first cheap stage and never examined further, so the detector spends its effort only on the few promising regions. OpenCV ships these cascades as ready-made XML files — the program loads `haarcascade_frontalface_default.xml` for faces and `haarcascade_eye.xml` for eyes — so we benefit from a model already trained on thousands of examples [2].

3.4 Multi-Scale Detection and Its Parameters

A face can appear large or small in an image depending on how close the person is, so the detector cannot look for just one size. Instead it scans the image repeatedly while shrinking it slightly each pass, a process called multi-scale detection. Two parameters control the trade-off. The scale factor sets how much the image shrinks between passes (a value like 1.1 means 10% smaller each time — smaller steps catch more faces but run slower). The minimum-neighbours value sets how many overlapping detections must agree before a region is accepted as a real face, which filters out flukes. In this project the face stage uses a scale factor of 1.1 with five required neighbours, while the eye stage inside each face uses a gentler 1.05 with only two neighbours, because eyes are smaller and we are already looking inside a confirmed face region.

3.5 Two-Stage Verification: Reducing False Positives

The single most important design idea in the whole project is verification. A face detector run on a busy, textured background will sometimes shout “face!” at something that is not a face at all — a false positive. If our sentry trusted that blindly, it would constantly stand down for nothing, or fire at the wrong moments.

The fix is a second opinion. For every region the face detector proposes, the program crops just that region and runs an eye detector inside it. A genuine human face will almost always contain detectable eyes; a random patch of background usually will not. Only a face that also contains at least one eye is accepted as a real human. This face-then-eyes pipeline is a simple but powerful way to make a noisy classical detector behave far more reliably, and it is the core of the decision logic.

Part II — Practical Part

4. Implementation and Results

This part describes how the theory above became a working program, walks through the structure of the code and the decision pipeline, presents the results, and discusses them.

4.1 Project Structure

Rather than one long script, the program is split into pieces with clear jobs:

- `main.py` — the entry point: it loops over the images, speaks the warning, runs detection, and asks for the result to be shown.
- `config.py` — a single configuration object holding all settings: the text-to-speech engine, the Pygame mixer, the dictionary of available Haar cascade files, and the paths to the images and sound assets.
- `tools/cascade_classifier.py` — a helper class wrapping OpenCV's cascades, with the face-and-eye detection logic and the drawing of boxes and circles.
- `tools/tools_image.py` — helpers for reading an image as grayscale and for presenting the result (showing the image, drawing FIRE!, and playing the right sound).
- `tools/check_requirement.py` — a self-installer that verifies each required library is present and installs the missing ones, including OS-specific extras.
- `detects_faces_webcam.py` — the optional real-time mode that runs the same detection on a live webcam feed.
- `assets/` — the input images and the two sound files (a tone and a gunfire clip).

Keeping configuration, detection, and presentation separate means, for instance, that swapping in a different cascade or a different sound is a one-line change in the config, and nothing else needs to know.

4.2 The Detection and Decision Pipeline

The heart of the program is the pipeline shown in Figure 1. Each input image is converted to grayscale, scanned for faces, and each candidate face is then checked for eyes. The single yes/no answer — did we find a face with eyes? — drives the final decision.

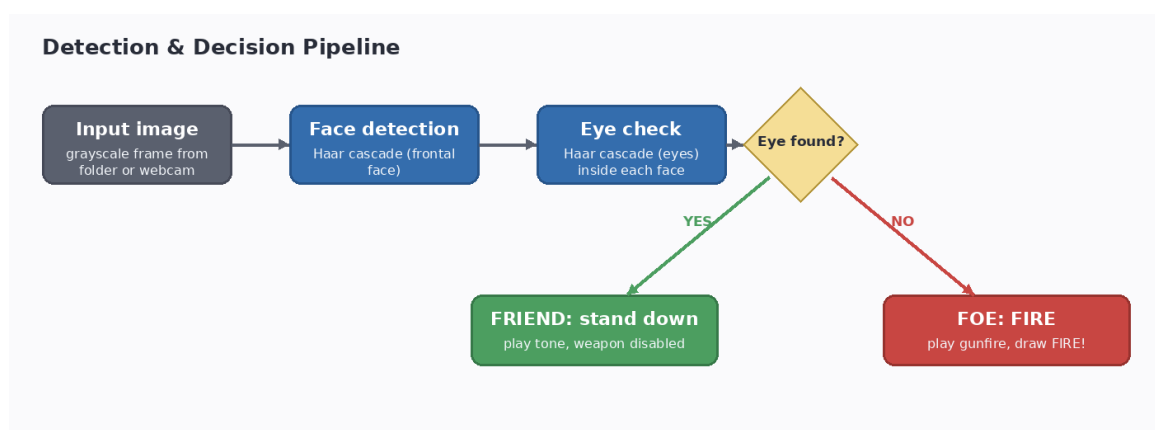


Figure 1. The detection and decision pipeline: input → face detection → eye verification → a binary friend-or-foe decision with audio-visual feedback.

In code, a customer-facing helper returns whether a verified human was found, and the main script simply inverts it: if no human is found, the weapon is “discharged”. This keeps the decision logic

readable — the messy detection details live in the helper, while the main loop reads almost like the English description of the rule.

4.3 Detection in Practice

When a face is found, the program draws a rectangle around it and, inside that rectangle, marks each detected eye with its own box and a circle — a clear visual confirmation of the two-stage verification at work. This also makes the system easy to debug: you can see exactly what it locked onto and why it decided the way it did.

4.4 Decision and Feedback

The feedback is deliberately theatrical, in keeping with the sentry theme. Before each image the text-to-speech engine speaks a Portal-style warning telling the subject to face the gun. If a verified human is detected, the system stands down and plays a calm tone. If no human is detected, it prints “FIRE!” across the centre of the image and plays a gunfire sound. Figure 2 shows the empty-corridor case, where there is no person to detect and the system therefore proceeds to the firing branch.

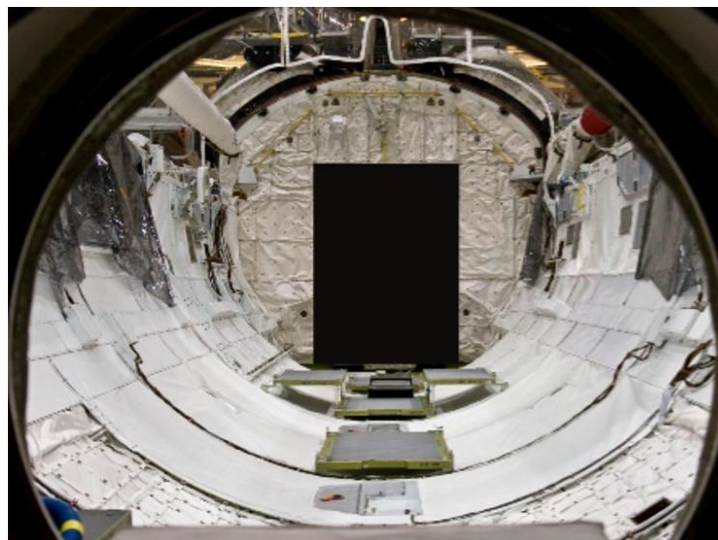


Figure 2. The empty-corridor test image (no person present). With no face to verify, the system takes the “fire” branch — the clearest demonstration of the negative case.

For privacy reasons this report does not reproduce the photographs of people used during testing; the empty scene above is sufficient to illustrate the negative branch, and Figure 1 illustrates the positive branch abstractly.

4.5 Real-Time Webcam Mode

Beyond the folder of still images, the project includes a real-time mode that opens the webcam and runs the same detect-and-decide loop on every frame until the user presses a key to quit. This reuses the exact same classifier and presentation helpers as the image mode, which is a direct payoff of keeping the code modular: the live mode is essentially the still-image logic placed inside a camera capture loop.

4.6 Results

The project works and meets its goals. On clear, well-lit, front-facing images the system reliably detects the face, confirms the eyes, and stands down; on the empty scene it correctly finds no

human and proceeds to fire. The two-stage verification noticeably reduces the false-positive faces that a face-only detector would otherwise report on the busy, metallic corridor backgrounds. The audio-visual feedback — spoken warning, tone or gunfire, and the on-screen FIRE! — all behave as intended, and the optional webcam mode runs the same logic live.

Just as importantly, the modular structure proved its worth: the same detection and presentation code serves both the still-image and the webcam paths, and the self-installing requirement check means the program sets up its own dependencies on a fresh machine.

4.7 Discussion

The decision that mattered most was the two-stage face-then-eyes verification. It is a small amount of extra code, but it is the difference between a jumpy detector that fires at shadows and one that behaves sensibly on cluttered backgrounds. The modular layout was the second good call: because configuration, detection, and presentation are separated, the live webcam mode came almost for free.

The honest limitations are exactly those of the classical Haar-cascade approach, and they are worth stating plainly. The detector is sensitive to face orientation — it is trained on upright, front-facing faces, so a tilted or profile view can slip past it. It is sensitive to lighting, since it reasons about brightness patterns. And it can still produce occasional false positives on complex backgrounds despite the eye check. These are not bugs in our code but inherent trade-offs of the technique, which is precisely why modern systems have largely moved to deep-learning detectors.

5. Summary

The goal of the project — to build a working, fun face-detection sentry while learning the fundamentals of classical computer vision — was achieved. The finished program detects faces, verifies them with eye detection to cut false positives, makes a clear friend-or-foe decision, and backs it up with spoken warnings, sounds, and on-screen feedback, in both a still-image mode and a live webcam mode.

More importantly for a learning project, the work delivered a genuine understanding of how classical detection works: images as brightness patterns, Haar-like features, the cascade that rejects easy negatives early, multi-scale scanning with its scale-factor and neighbour parameters, and — the key idea — verifying a detection with a second, independent check. Each of these started as an abstract concept from the literature and ended up as a concrete, inspectable part of the program.

If we were to continue, the path is clear and is reflected in the project's own roadmap: move from still images to robust real-time video, replace the Haar cascades with a deep-learning detector such as a convolutional neural network for far better accuracy across poses and lighting, add tracking so a detected person is followed smoothly across frames, and broaden the classification beyond "face or not" to handle masks, profiles, and other objects. None of these require throwing away the current structure, which we take as a sign that the foundation was sound. On a subjective note, the most satisfying moment was watching the little sentry confidently announce FIRE! at an empty corridor and stand down politely for a real face — that was when a pile of computer-vision theory finally felt like a working machine with a personality.

6. Bibliography

- [1] P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” Proc. IEEE CVPR, 2001. Overview available: https://en.wikipedia.org/wiki/Viola%E2%80%93Jones_object_detection_framework
- [2] OpenCV, “Cascade Classifier” and “Face Detection using Haar Cascades” (official documentation and pre-trained models). Available: https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- [3] pyttsx3, “Text-to-speech library for Python” (documentation). Available: <https://pyttsx3.readthedocs.io/>
- [4] Wikipedia contributors, “Haar-like feature” and “Viola–Jones object detection framework,” Wikipedia, The Free Encyclopedia. Available: https://en.wikipedia.org/wiki/Haar-like_feature
- [5] titou4n, “IdentifyingFriendOrFoe” (project source repository). Available: <https://github.com/titou4n/IdentifyingFriendOrFoe>